

Serverless Image Processing Pipeline

Project 2 • Serverless Architecture • Terraform IaC • Python 3.11

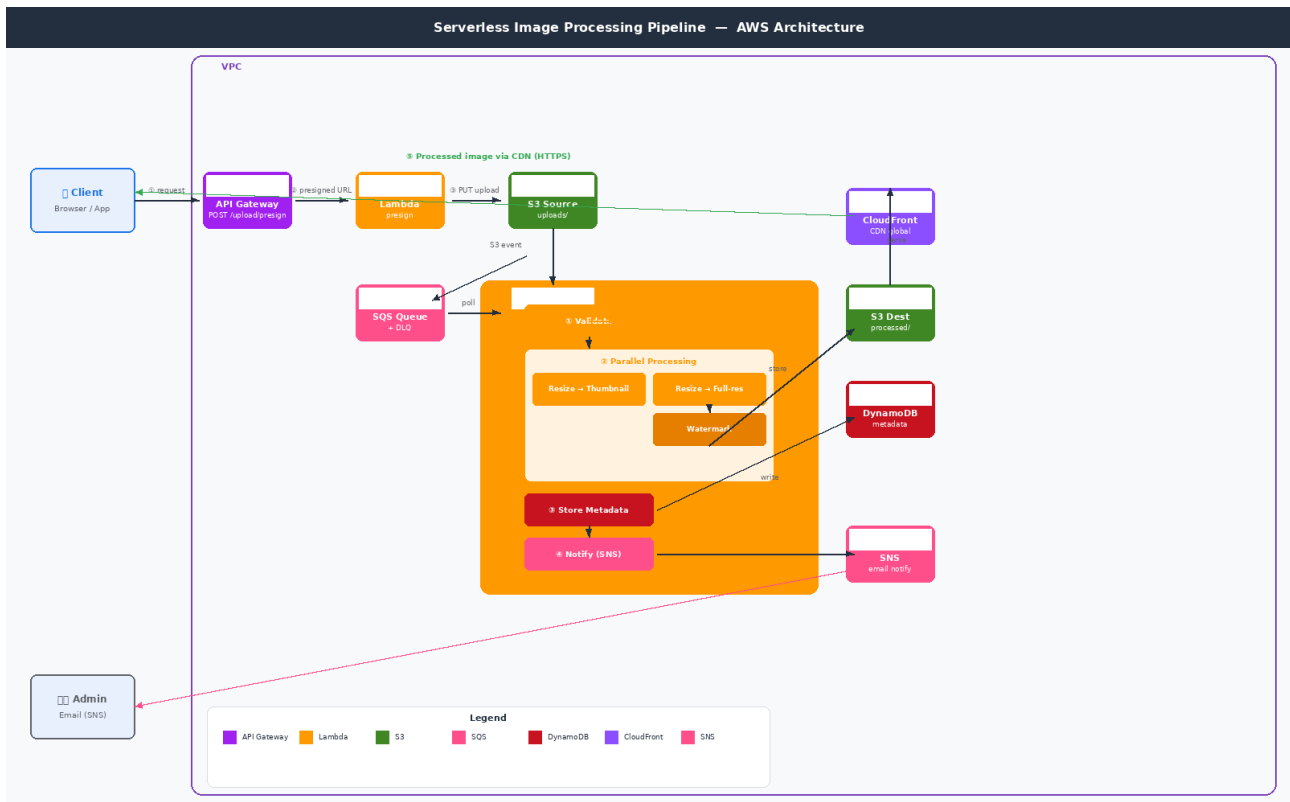
AWS Serverless

Terraform IaC

Python 3.11

Event-Driven

Architecture Diagram



1. Project Overview

A fully serverless, event-driven image processing pipeline built on AWS. Clients upload images via a pre-signed S3 URL, which triggers an automated workflow: images are validated, resized to thumbnail and full-resolution outputs, watermarked, and stored — all without managing any servers. Results are served globally through CloudFront CDN.

Attribute	Value
Architecture	Serverless (no EC2/ECS)
IaC Tool	Terraform >= 1.5.0
Runtime	Python 3.11 (Lambda)
Image Library	Pillow (Lambda Layer)

Workflow Engine	AWS Step Functions (Standard)
CDN	Amazon CloudFront + OAC
Database	DynamoDB (PAY_PER_REQUEST)
Notifications	SNS → Email

2. AWS Services & Responsibilities

Service	Role	Key Config
S3 (×2)	Source + destination buckets	Lifecycle rules, Event notifications, OAC
SQS + DLQ	Decouple events from processing	Long polling 20s, maxReceiveCount=3
Lambda (×6)	Presign, validate, resize, watermark, store, notify	Python 3.11, Pillow Layer, X-Ray
Step Functions	Orchestrate multi-step workflow	Standard type, parallel branches, retries
API Gateway	Pre-signed URL endpoint	HTTP API v2, POST /upload/presign
DynamoDB	Image metadata store	PAY_PER_REQUEST, GSI on status, PITR
CloudFront	Global CDN for processed images	OAC (no public S3), HTTPS redirect
SNS	Job completion/failure notifications	Email subscription, KMS encryption

3. Step Functions Workflow

The workflow is a Standard state machine with 4 sequential stages and 1 parallel stage:

① Validate	Checks image content-type (JPEG/PNG/WebP) and file size (max 20MB). Invalid images update DynamoDB to INVALID status and skip the rest of the workflow.
② Parallel Processing	Two branches execute concurrently: (a) Resize to 200×200 thumbnail; (b) Resize to max 1920×1080 full-res, then apply semi-transparent text watermark.
③ Store Metadata	Writes all output S3 keys, dimensions, timestamps, and COMPLETED status to DynamoDB.
④ Notify	Publishes a success or failure message to SNS → subscriber email.
Error Handling	Each Lambda task has 3 retries with exponential backoff. Any unhandled error routes to NotifyFailure → PipelineFailed state.

4. Deployment Instructions

Step 1 — Build Pillow Lambda Layer

```
mkdir -p layers/python pip install pillow -t layers/python/ --platform  
manylinux2014_x86_64 \ --implementation cp --python-version 3.11  
--only-binary=:all: cd layers && zip -r pillow_layer.zip python/ && cd ..
```

Step 2 — Configure Variables

```
cp terraform.tfvars.example terraform.tfvars # Edit: set notification_email,  
project_name, environment
```

Step 3 — Deploy

```
terraform init terraform plan terraform apply
```

Step 4 — Confirm SNS Email

Check your inbox and confirm the AWS SNS subscription email to start receiving notifications.

5. Key Design Decisions

Decision	Rationale
S3 → SQS (not direct to Lambda)	SQS enables retries, DLQ for failures, back-pressure, and full decoupling of ingestion from processing.
Lambda Layer for Pillow	Keeps each function's deployment package small. The layer is shared across resize and watermark functions.
Step Functions STANDARD type	Provides full execution history, per-step retry logic, parallel branches, and easy debugging via CloudWatch.
Parallel resize branches	Thumbnail and full-res are independent — processing both concurrently cuts total workflow time in half.
CloudFront + OAC	The destination S3 bucket is fully private. CloudFront authenticates via SigV4 Origin Access Control.
DynamoDB PAY_PER_REQUEST	No capacity planning required for variable or unpredictable image upload volumes.
SQS long polling (20s wait time)	Reduces the number of empty ReceiveMessage API calls, lowering cost and latency on message processing.

6. DynamoDB Schema

Attribute	Type	Description
image_id (PK)	String	UUID generated at upload time
created_at (SK)	String	ISO 8601 timestamp of upload
status	String	PENDING → PROCESSING → COMPLETED / FAILED / INVALID
source_key	String	S3 key in source bucket (uploads/)
thumbnail_key	String	S3 key in destination bucket (processed/thumbnails/)
fullres_key	String	S3 key in destination bucket (processed/fullres/)
content_type	String	Image MIME type (image/jpeg, image/png, image/webp)
file_size	Number	Original file size in bytes
completed_at	String	Processing completion timestamp (ISO 8601)
expires_at	Number	TTL — Unix timestamp for automatic DynamoDB cleanup

GSI: status-index — allows querying all images by status (e.g. find all FAILED jobs).

7. Learning Outcomes

- Design event-driven architectures using S3 event notifications and SQS decoupling.

- Understand why SQS improves resilience: retries, DLQ, and back-pressure without code changes.
- Package and use Lambda Layers to share large dependencies (Pillow) across multiple functions.
- Orchestrate multi-step serverless workflows with Step Functions, including parallel branches and error handling.
- Apply S3 lifecycle policies to automatically transition objects: Standard → Standard-IA → Glacier → Expire.
- Serve processed content through CloudFront using Origin Access Control (OAC) for secure private S3 access.

Infrastructure as Code (Terraform) | All resources tagged with Project / Environment / ManagedBy | force_destroy=true on S3 buckets for dev teardown — remove for production